

Лабораторная работа 7

Введение в работу с данными

Ланцова Яна Игоревна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Задание 1	12
3.2	Задание 2	15
3.3	Задание 3	17
4	Выводы	20

Список иллюстраций

3.1	Считывание данных	7
3.2	Считывание данных	7
3.3	Запись данных в файл	8
3.4	Словари	8
3.5	DataFrames	8
3.6	DataFrames	9
3.7	RDatasets	9
3.8	RDatasets	9
3.9	Работа с переменными отсутствующего типа (MissingValues)	10
3.10	Кластеризация данных. Метод k-средних	10
3.11	Кластеризация данных. Метод k-средних	10
3.12	Кластеризация данных. Метод k-средних	11
3.13	Обработка данных. Линейная регрессия	11
3.14	Обработка данных. Линейная регрессия	11
3.15	Кластеризация	12
3.16	Загрузка и просмотр данных	12
3.17	Подготовка данных	13
3.18	Создание диаграммы	13
3.19	Диаграмма	13
3.20	Создание диаграммы	14
3.21	Диаграмма	14
3.22	Создание итогового Dataframe	14
3.23	Создание итогового Dataframe	15
3.24	Вывод информации о кластерах	15
3.25	Генерация данных и ручная оценка	16
3.26	Сопоставление результатов	16
3.27	Построение линии регрессии	16
3.28	График регрессии	17
3.29	Описание задания	18
3.30	Модель ценообразования биномиальных опционов	18
3.31	График траекторий цены акции	18
3.32	График для различных траекторий цены	19
3.33	Параллельное вычисление траекторий	19

Список таблиц

1 Цель работы

Основной целью работы является освоение специализированных пакетов Julia для обработки данных.

2 Задание

1. Используя JupyterLab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы.

3 Выполнение лабораторной работы

Выполним примеры из лабораторной работы (рис. 3.1 - 3.10).

```
[2]: # Считывание данных и их запись в структуру:
P = CSV.File("/Users/yana/downloads/programminglanguages.csv") |> DataFrame

# 73x2 DataFrame

48 rows omitted
```

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN
7	1958	LISP
8	1958	ALGOL 58
9	1959	FACT
10	1959	COBOL
11	1959	RPG
12	1962	APL
13	1962	Simula
:	:	:

Рис. 3.1: Считывание данных

```
[4]: # Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

[4]: language_created_year (generic function with 1 method)

[5]: # Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")
# Пример вызова функции и определение даты создания языка Julia:
language_created_year(P,"Julia")

[5]: 2012

[9]: # Функция определения по названию языка программирования
# года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
    return P[loc,1]
end

# Пример вызова функции и определение даты создания языка julia:
language_created_year_v2(P,"julia")

[9]: 2012

[10]: # Построчное считывание данных с указанием разделителя:
Tx = readlm("~/Users/yana/Downloads/programminglanguages.csv", ',')

[10]: 74x2 Matrix{Any}:
"year"  "Language"
1951    "Regional Assembly Language"
1952    "Autocode"
1954    "IPL"
1955    "FLOW-MATIC"
```

Рис. 3.2: Считывание данных

```
[11]: # Пример записи данных в текстовый файл с разделителем ',';
      writeln("programming_languages_data.txt", Tx, ',');
      # Пример записи данных в текстовый файл с разделителем '-';
      writeln("programming_languages_data2.txt", Tx, '-');

[12]: # Построчное считывание данных с указанием разделителя;
      P_new_delim = readln("programming_languages_data2.txt", '-');

[12]: 74x2 Matrix(Any):
      "year"      "language"
      1951        "Regional Assembly Language"
      1952        "Autocode"
      1954        "IPL"
      1955        "FLOW-MATIC"
      1957        "FORTRAN"
      1957        "COMTRAN"
      1958        "LISP"
      1958        "ALGOL 58"
      1959        "FACT"
      1959        "COBOL"
      1959        "RPG"
      1962        "APL"
      2003        "Scala"
      2005        "F#"
      2006        "PowerShell"
      2007        "Clojure"
      2009        "Go"
      2010        "Rust"
      2011        "Dart"
      2011        "Kotlin"
      2011        "Red"
```

Рис. 3.3: Запись данных в файл

Словари

```
[13]: # Инициализация словаря:
      dict = Dict{Integer,Vector{String}}{()}

[13]: Dict{Integer, Vector{String}}{()}

[14]: # Инициализация словаря:
      dict2 = Dict{String, Integer}{}

[14]: Dict{String, Integer}{}

[16]: # Заполнение словаря данными:
      for i = 1:size(P,1)
          year, lang = P[i,:]
          if year in keys(dict)
              dict[year] = push!(dict[year], lang)
          else
              dict[year] = [lang]
          end
      end

[17]: dict[2003]

[17]: 4-element Vector{String}:
      "Groovy"
      "Scala"
      "Groovy"
      "Scala"
```

Рис. 3.4: Словари

DataFrames

```
[18]: # Подгружаем пакет DataFrames:
      using DataFrames
      # Задаем переменную со структурой DataFrame:
      df = DataFrame(year = P[:,1], language = P[:,2])

[18]: 73x2 DataFrame
      Row  year  language
      Int64 String31
      1  1951  Regional Assembly Language
      2  1952  Autocode
      3  1954  IPL
      4  1955  FLOW-MATIC
      5  1957  FORTRAN
      6  1957  COMTRAN
      7  1958  LISP
      8  1958  ALGOL 58
      9  1959  FACT
      10 1959  COBOL
      11 1959  RPG
      12 1962  APL
      13 1962  Simula
      :      :
      62 2003  Scala
      63 2005  F#
```

Рис. 3.5: DataFrames


```
[20]: # Получение статистических сведений о фрейме:
describe(df)

[20]: 2x7 DataFrame
Row variable mean min median max nmissing eltype
Symbol Union... Any Union... Any Int64 DataType
1 year 1982.99 1951 1986.0 2014 0 Int64
2 language ALGOL 58 dBase III 0 String31
```

Рис. 3.6: DataFrames

RDatasets

```
[21]: # Подгружаем пакет RDatasets:
using RDatasets
# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")

[21]: 150x5 DataFrame 125 rows omitted
Row Sepal.Length Sepal.Width Petal.Length Petal.Width Species
Float64 Float64 Float64 Float64 Cat...
1 5.1 3.5 1.4 0.2 setosa
2 4.9 3.0 1.4 0.2 setosa
3 4.7 3.2 1.3 0.2 setosa
4 4.6 3.1 1.5 0.2 setosa
5 5.0 3.6 1.4 0.2 setosa
6 5.4 3.9 1.7 0.4 setosa
7 4.6 3.4 1.4 0.3 setosa
8 5.0 3.4 1.5 0.2 setosa
9 4.4 2.9 1.4 0.2 setosa
10 4.9 3.1 1.5 0.1 setosa
11 5.4 3.7 1.5 0.2 setosa
12 4.8 3.4 1.6 0.2 setosa
13 4.8 3.0 1.4 0.1 setosa
: : : : : :
```

Рис. 3.7: RDatasets

```
[22]: # Определения типа переменной:
typeof(iris)

[22]: DataFrame

[23]: describe(iris)

[23]: 5x7 DataFrame
Row variable mean min median max nmissing eltype
Symbol Union... Any Union... Any Int64 DataType
1 Sepal.Length 5.84333 4.3 5.8 7.9 0 Float64
2 Sepal.Width 3.05733 2.0 3.0 4.4 0 Float64
3 Petal.Length 3.758 1.0 4.35 6.9 0 Float64
4 Petal.Width 1.19933 0.1 1.3 2.5 0 Float64
5 Species setosa virginica 0 CategoricalValue{String, UInt8}
```

Рис. 3.8: RDatasets

Работа с переменными отсутствующего типа (Missing Values)

```
[24]: # Отсутствующий тип:
a = missing
typeof(a)

[24]: Missing

[25]: # Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]

[25]: 4-element Vector{Union{Missing, Int64}}:
missing
 47
 22
105

[26]: # Определение типа переменной:
typeof(calories)

[26]: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})

[28]: # Получаем пакет Statistics:
using Statistics
# Определение среднего значения:
mean(calories)
# Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

[28]: 58.0
```

Рис. 3.9: Работа с переменными отсутствующего типа (Missing Values)

Кластеризация данных. Метод k-средних

```
[36]: # Загрузка данных:
houses = CSV.File("/Users/yana/Downloads/houses.csv") |> DataFrame

[36]: 985x12 DataFrame                                     960 rows omitted
   Row  street                city  zip  state  beds  baths  sq__ft  type  sale_date  price  latitude  longitude
   String String15             Int64 String3 Int64 Int64 Int64 String15 String31 Int64 Float64 Float64
1  3526 HIGH ST      SACRAMENTO  95838  CA     2     1    836  Residential  Wed May 21 00:00:00 EDT 2008  59222  38.6319  -121.435
2  51 OMAHA CT       SACRAMENTO  95823  CA     3     1   1167  Residential  Wed May 21 00:00:00 EDT 2008  68212  38.4789  -121.431
3  2796 BRANCH ST    SACRAMENTO  95815  CA     2     1    796  Residential  Wed May 21 00:00:00 EDT 2008  68880  38.6183  -121.444
4  2805 JANETTE WAY  SACRAMENTO  95815  CA     2     1    852  Residential  Wed May 21 00:00:00 EDT 2008  69307  38.6168  -121.439
5  6001 MCMAHON DR   SACRAMENTO  95824  CA     2     1    797  Residential  Wed May 21 00:00:00 EDT 2008  81900  38.5195  -121.436
6  5828 PEPPERMILL CT SACRAMENTO  95841  CA     3     1   1122  Condo       Wed May 21 00:00:00 EDT 2008  89921  38.6626  -121.328
7  6048 OGDEN NASH WAY SACRAMENTO  95842  CA     3     2   1104  Residential  Wed May 21 00:00:00 EDT 2008  90895  38.6817  -121.352
8  2561 19TH AVE     SACRAMENTO  95820  CA     3     1   1177  Residential  Wed May 21 00:00:00 EDT 2008  91002  38.5351  -121.481
9  11150 TRINITY RIVER DR Unit 114 RANCHO CORDOVA 95670  CA     2     2    941  Condo       Wed May 21 00:00:00 EDT 2008  94905  38.6212  -121.271
```

Рис. 3.10: Кластеризация данных. Метод k-средних

```
[37]: # Построение графика:
using Plots
plot(size=(500,500),leg=false)
x = houses[:, :sq__ft]
y = houses[:, :price]
scatter(x, y, markersize=3)
```

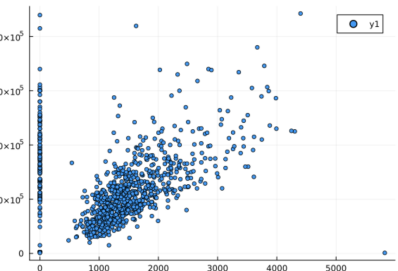


Рис. 3.11: Кластеризация данных. Метод k-средних

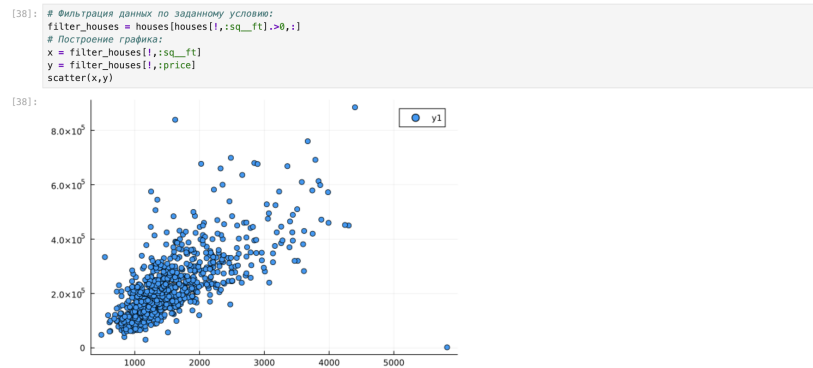


Рис. 3.12: Кластеризация данных. Метод k-средних

Обработка данных. Линейная регрессия

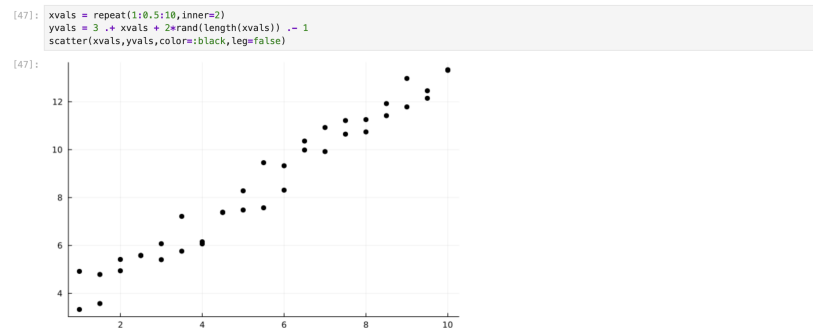


Рис. 3.13: Обработка данных. Линейная регрессия

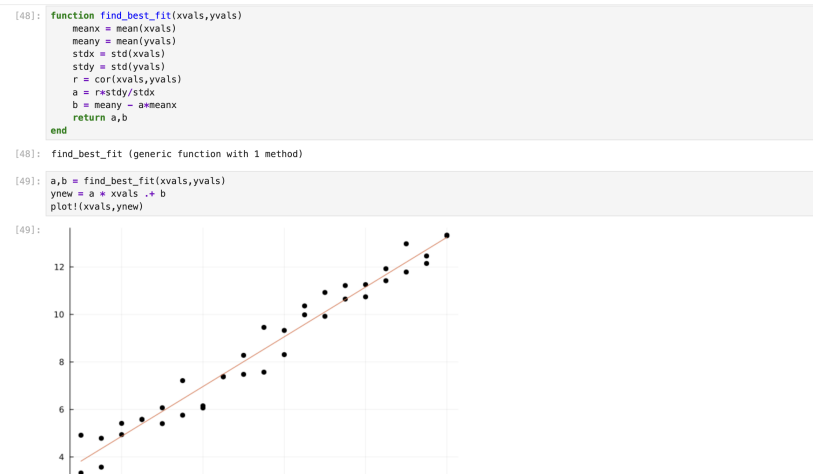


Рис. 3.14: Обработка данных. Линейная регрессия

Теперь перейдем к заданиям для самостоятельного выполнения

3.1 Задание 1

Загрузим

using RDatasets

iris = dataset("datasets", "iris")

Используем Clustering.jl для кластеризации на основе k-средних. Сделаем точечную диаграмму полученных кластеров. Для этого проиндексируем фрейм данных, преобразуем его в массив и транспонируем (рис. 3.15 - 3.24).

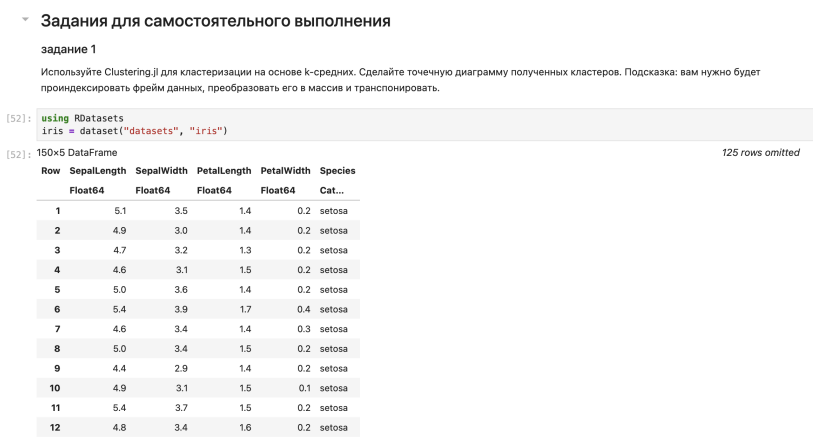


Рис. 3.15: Кластеризация

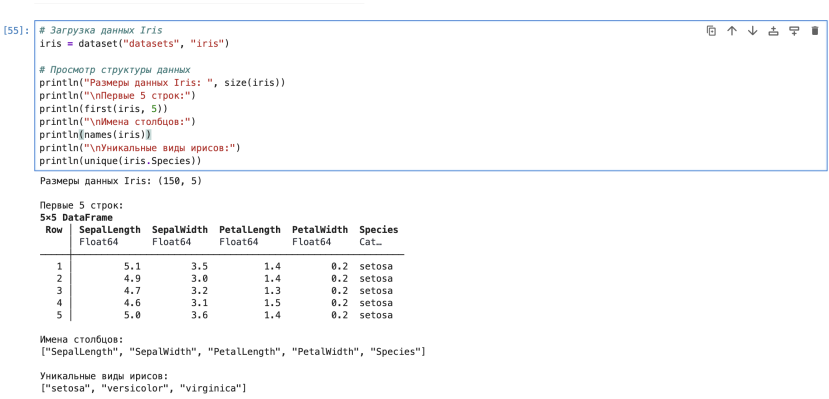


Рис. 3.16: Загрузка и просмотр данных

```
[56]: # Подготовка данных для кластеризации
# Берем только числовые признаки (первые 4 колонки)
iris_features = iris[:, 1:4]

# Преобразуем DataFrame в матрицу и транспонируем
# (Clustering.jl ожидает данные в виде матрицы признаков x наблюдений)
iris_matrix = Matrix(iris_features)'
println("\nРазмерность матрицы признаков: ", size(iris_matrix))

Размерность матрицы признаков: (4, 150)

[57]: # Выполняем кластеризацию методом k-средних
k = 3 # Мы знаем, что в данных 3 вида ирисов
result = kmeans(iris_matrix, k)

# Получаем метки кластеров
cluster_labels = assignments(result)

# Получаем центры кластеров
centroids = result.centers

println("\nКластеризация завершена:")
println("Количество кластеров: ", k)
println("Размеры каждого кластера: ", counts(result))
println("Итераций выполнено: ", result.iterations)
println("Общая стоимость (сумма квадратов расстояний): ", round(result.totalcost, digits=3))

Кластеризация завершена:
Количество кластеров: 3
Размеры каждого кластера: [50, 39, 61]
Итераций выполнено: 7
Общая стоимость (сумма квадратов расстояний): 78.856
```

Рис. 3.17: Подготовка данных

```
[58]: # Создаем точечную диаграмму (scatter plot)
# Используем два наиболее информативных признака для визуализации
# SepalLength и SepalWidth (длина и ширина чашелистика)

# Создаем цветовую палитру для кластеров
colors = [:red, :blue, :green, :orange, :purple] # больше цветов на всякий случай
cluster_colors = [colors[label] for label in cluster_labels]

# Создаем точечную диаграмму
scatter(iris.SepalLength, iris.SepalWidth,
        color=cluster_colors,
        markerstrokewidth=0.5,
        markersize=6,
        alpha=0.7,
        legend=false,
        xlabel="Sepal Length (cm)",
        ylabel="Sepal Width (cm)",
        title="Кластеризация Iris методом k-средних (k=3)",
        grid=true,
        size=(800, 600))
```

Рис. 3.18: Создание диаграммы

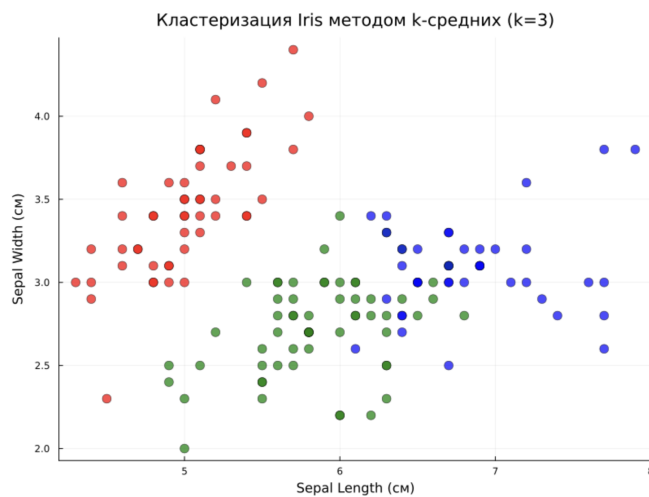


Рис. 3.19: Диаграмма

```

: # Добавляем центры кластеров
for i in 1:k
    scatter!(centroids[i, 1], [centroids[i, 2], i],
            color=colors[i],
            markersize=12,
            marker=:star,
            markerstrokewidth=2,
            label="Центроид кластера $i")
end

# Добавляем легенду для центроидов
plot!(legend=:topright)

```

Рис. 3.20: Создание диаграммы

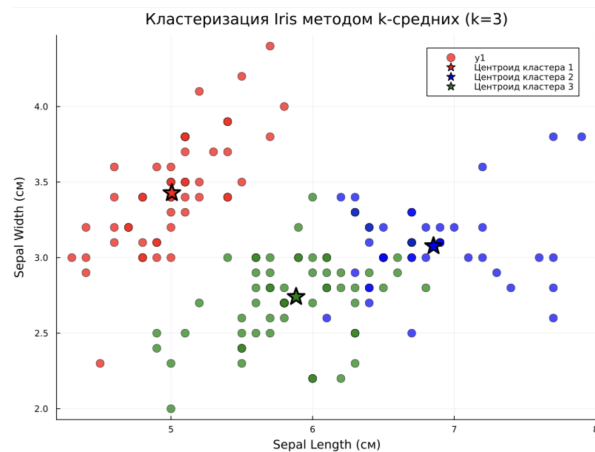


Рис. 3.21: Диаграмма

```

[61]: # Создаем DataFrame с результатами кластеризации
using DataFrames
iris_with_clusters = copy(iris)
iris_with_clusters[:, :Cluster] = cluster_labels

# Функция для создания матрицы парных диаграмм
function pairplot_matrix(df)
    features = ["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"]
    n = length(features)

    plot_array = []

    for i in 1:n
        for j in 1:n
            if i == j
                # Диагональ - гистограмма распределения
                p = histogram(df[:, features[i]],
                            color=:gray, alpha=0.7,
                            legend=false,
                            title=features[i],
                            grid=false)
            else
                # Внедиагональные элементы - scatter plots
                p = scatter(df[:, features[j]], df[:, features[i]],
                            color=cluster_colors,
                            markerstrokewidth=0,
                            markersize=4,
                            alpha=0.6,
                            legend=false,
                            grid=false)
            end
            push!(plot_array, p)
        end
    end

    return plot(plot_array..., layout=(n, n), size=(1000, 1000))
end

```

Рис. 3.22: Создание итогового DataFrame

```

# Создаем и сохраняем матрицу парных диаграмм
pairplot = pairplot_matrix(iris_with_clusters)
savefig(pairplot, "iris_pairplot_matrix.png")
println("Матрица парных диаграмм сохранена как 'iris_pairplot_matrix.png'")

# Сравнение кластеров с реальными видами ирисов
println("\nСравнение кластеров с реальными видами:")

# Создаем таблицу сопоставленности
real_species = iris.Species
cluster_df = DataFrame(RealSpecies=real_species, Cluster=cluster_labels)

Матрица парных диаграмм сохранена как 'iris_pairplot_matrix.png'

Сравнение кластеров с реальными видами:
[61]: 150x2 DataFrame

```

Row	RealSpecies	Cluster
Cat...	Int64	
1	setosa	1
2	setosa	1
3	setosa	1
4	setosa	1
5	setosa	1
6	setosa	1
7	setosa	1
8	setosa	1
9	setosa	1
10	setosa	1
11	setosa	1

125 rows omitted

Рис. 3.23: Создание итогового DataFrame

```

]: # Расчет инерции (суммы квадратов расстояний до центроидов)
println("\nИнформация о кластерах:")
for i in 1:k
    cluster_points = iris_matrix[:, cluster_labels .== i]
    cluster_size = size(cluster_points, 2)

    if cluster_size > 0
        # Среднее расстояние до центроида
        distances = [sqrt(sum((cluster_points[:, j] - centroids[:, i]).^2))
                     for j in 1:cluster_size]
        avg_distance = round(mean(distances), digits=3)
        std_distance = round(std(distances), digits=3)

        println("Кластер $i: $cluster_size образцов, среднее расстояние до центроида: $avg_distance ± $std_distance")
    end
end

Информация о кластерах:
Кластер 1: 50 образцов, среднее расстояние до центроида: 0.482 ± 0.269
Кластер 2: 39 образцов, среднее расстояние до центроида: 0.732 ± 0.345
Кластер 3: 61 образцов, среднее расстояние до центроида: 0.731 ± 0.388

```

Рис. 3.24: Вывод информации о кластерах

3.2 Задание 2

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ и $\text{randn}(N, 3)$, массив результатов в $N \times 1$, регрессионная зависимость является линейной. Найдём МНК-оценку для линейной модели (рис. 3.25 - 3.26):

- Сравним свои результаты с результатами использования `llsq` из `MultivariateStats.jl`.
- Сравним свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`.

задание 2

часть 1

Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ `randn(N, 3)`, массив результатов $N \times 1$, регрессионная зависимость является линейной. Найдите МНК-оценку для линейной модели.

– Сравните свои результаты с результатами использования `llsq` из `MultivariateStats.jl` (просмотрите документацию).

– Сравните свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`. Подсказка. Создайте матрицу данных X_2 , которая добавляет столбец единиц в начало матрицы данных, и решите систему линейных уравнений.

Объясните с помощью теоретических выкладок.

```
3): # Генерация данных
N = 1000
X = randn(N, 3)
a0 = rand(3)
y = X * a0 + 0.1 * randn(N)

# МНК-оценка вручную (с добавлением столбца единиц для свободного члена)
X2 = hcat(ones(N), X) # Добавляем столбец единиц – это важно для интерцепта!
a_mnk = (X2' * X2) \ (X2' * y) # Решение системы (X'X)a = X'y
println("МНК-оценка (вручную): ", a_mnk)

# Сравнение с MultivariateStats.jl (функция 'llsq')
a_llsq = llsq(X, y, bias=true) # bias=true добавляет константу (столбец единиц)
println("МНК-оценка (MultivariateStats.jl, llsq): ", a_llsq)

МНК-оценка (вручную): [6.0337110182196254e-5, 0.4716359063598927, 0.48730003271456, 0.3845485710561637]
МНК-оценка (MultivariateStats.jl, llsq): [0.47163590635989283, 0.48730003271455996, 0.38454857105616375, 6.033711018219987e-5]
```

Рис. 3.25: Генерация данных и ручная оценка

```
: # Сравнение с GLM.jl (регулярная линейная регрессия)
# Создаем DataFrame для GLM (если не установлен – установите пакет DataFrames)
using DataFrames
df = DataFrame(X1=X[:,1], X2=X[:,2], X3=X[:,3], y=y)
model_glm = lm(@formula(y ~ X1 + X2 + X3), df)
a_glm = coef(model_glm)
println("МНК-оценка (GLM.jl): ", a_glm)

# Проверка совпадения результатов
println("\nСравнение результатов:")
println("Разница между ручным МНК и llsq: ", norm(a_mnk - a_llsq))
println("Разница между ручным МНК и GLM: ", norm(a_mnk - a_glm))

# Теоретическое объяснение
println("""
Матрица X2 = [1 X] позволяет оценить модель y = β₀ + β₁x₁ + β₂x₂ + β₃x₃.
Теоретически, МНК-оценка находится как a = (X2'X2)⁻¹X2'y.
Это стандартный метод решения overdetermined системы уравнений.
""")

МНК-оценка (GLM.jl): [6.033711018220072e-5, 0.47163590635989283, 0.48730003271455996, 0.38454857105616375]

Сравнение результатов:
Разница между ручным МНК и llsq: 0.6172665123619459
Разница между ручным МНК и GLM: 1.360473030665105e-16
Матрица X2 = [1 X] позволяет оценить модель y = β₀ + β₁x₁ + β₂x₂ + β₃x₃.
Теоретически, МНК-оценка находится как a = (X2'X2)⁻¹X2'y.
Это стандартный метод решения overdetermined системы уравнений.
```

Рис. 3.26: Сопоставление результатов

Найдем линию регрессии, используя данные (X, y) . Построим график (X, y) , используя точечный график. Добавим линию регрессии, используя `abline`!. Добавим заголовок «График регрессии» и подпишем оси x и y (рис. 3.27 - 3.28):

```
часть 2

Найдите линию регрессии, используя данные  $(X, y)$ . Постройте график  $(X, y)$ , используя точечный график. Добавьте линию регрессии, используя abline!.
Добавьте заголовок «График регрессии» и подпишите оси  $x$  и  $y$ 

|: # Генерация данных
X = rand(100)
y = 2 * X + 0.1 * randn(100)

# Регрессия (одна переменная, с константой)
X2 = hcat(ones(100), X)
a_mnk_id = (X2' * X2) \ (X2' * y)
println("Оценки коэффициентов (одномерная регрессия): ", a_mnk_id)

# Построение графика
p = scatter(X, y, label="Данные", title="График регрессии", xlabel="x", ylabel="y")

# Рассчитываем значения линии регрессии
x_line = range(minimum(X), maximum(X), length=100)
y_line = a_mnk_id[1] + a_mnk_id[2] * x_line

# Добавляем линию регрессии
plot!(p, x_line, y_line, label="Линия регрессии", color=:red, linewidth=2)

# Отображаем график
display(p)

Оценки коэффициентов (одномерная регрессия): [0.012201122293427695, 1.9847617324640696]
```

Рис. 3.27: Построение линии регрессии

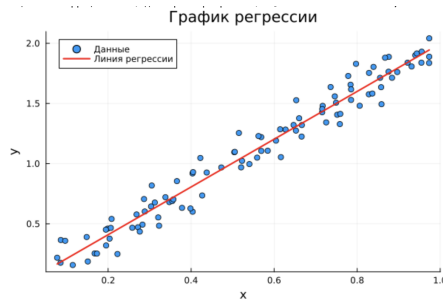


Рис. 3.28: График регрессии

3.3 Задание 3

Построим траекторию возможных цен на акции:

- S – начальная цена акции;
- T – длина биномиального дерева в годах;
- n – количество периодов;
- $h = Tn$ – длина одного периода;
- σ – волатильность акции;
- r – годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$;

Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуем построить траекторию курса акций (рис. 3.29 - 3.31):

задание 3

Описание модели ценообразования биномиальных опционов можно найти на стр. https://en.wikipedia.org/wiki/Binomial_options_pricing_model.

Постройте траекторию возможных цен на акции:

- S — начальная цена акции;
- T — длина биномиального дерева в годах;
- n — количество периодов;
- $h = T/n$ — длина одного периода;
- σ — волатильность акции;
- r — годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;

$$p^* = (\exp(rh) - d) / (u - d)$$

Учтите, что

- Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.
- Создайте функцию `createPath` ($S :: \text{Float64}$, $r :: \text{Float64}$, $\sigma :: \text{Float64}$, $T :: \text{Float64}$, $n :: \text{Int64}$), которая создает траекторию цены акции с учетом начальных параметров. Используйте `createPath`, чтобы создать 10 разных траекторий и построить их все на одном графике.
- Распараллельте генерацию траектории. Можете использовать `Threads.@threads`, `rmmap` и `@parallel`.
- Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1.

Рис. 3.29: Описание задания

```
[78]: function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
    h = T / n
    u = exp(r * h + sigma * sqrt(h))
    d = exp(r * h - sigma * sqrt(h))
    p_star = (exp(r * h) - d) / (u - d)

    # Генерируем траекторию
    path = Vector{Float64}(undef, n+1)
    path[1] = S

    for i in 2:(n+1)
        if rand() < p_star
            path[i] = path[i-1] * u
        else
            path[i] = path[i-1] * d
        end
    end

    return path
end

# Параметры
S = 100.0
T = 1.0
n = 10000
sigma = 0.3
r = 0.08;
```

Рис. 3.30: Модель ценообразования биномиальных опционов

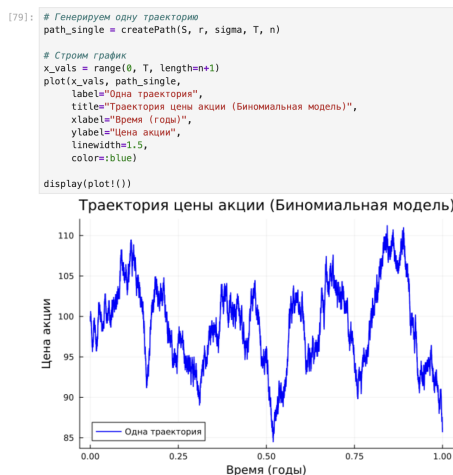


Рис. 3.31: График траекторий цены акции

Создадим функцию `createPath` ($S :: \text{Float64}$, $r :: \text{Float64}$, $\sigma :: \text{Float64}$, $T :: \text{Float64}$, $n :: \text{Int64}$), которая создает траекторию цены акции с учетом

начальных параметров. Используем `createPath`, чтобы создать 10 разных траекторий и построим их все на одном графике (рис. 3.32 - 3.33):

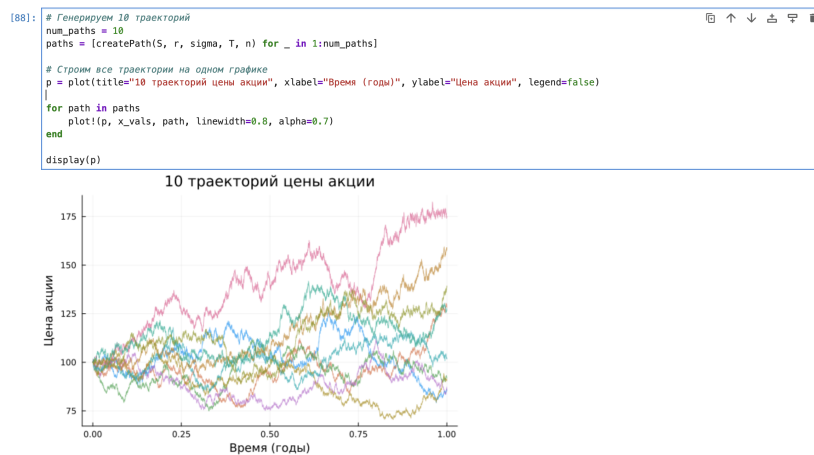


Рис. 3.32: График для различных траекторий цены

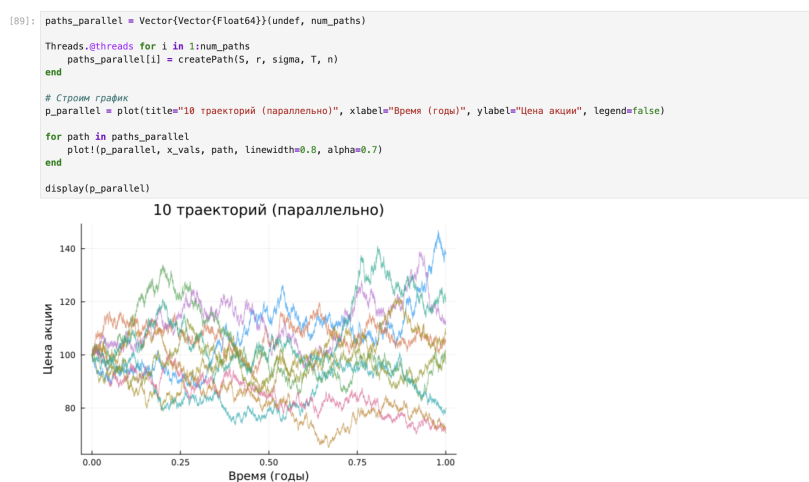


Рис. 3.33: Параллельное вычисление траекторий

4 Выводы

В результате выполнения данной лабораторной работы я освоила специализированные пакеты Julia для обработки данных.